

САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Фронт-энд разработка

Отчёт

Домашняя работа 5

Выполнила:
Вдовина Анастасия
Группа J3210

Проверил:
Добряков Д. И.

Санкт-Петербург
2026 г.

Задача

Изучить основные команды пакетного менеджера `npm` и собрать проект на `Vue` по мануалу курса. Приложение сделано по теме курсового проекта: список мероприятий `EventTix`, который загружается с сервера, ищется по строке и пополняется через форму.

Ход работы

Этап 1 — `npm` и создание проекта

Проект создан командой `npm init vue@latest`, зависимости поставлены через `npm install`. В секции `scripts` прописан запуск `start` на порту 8080, как указано в мануале. Данные отдаёт `json-server`, он поднимается скриптом `npm run api` на порту 3000.

Установленные пакеты: `axios` для запросов, `pinia` и `pinia-persists` для хранения, `vue-router` для маршрутов, `bootstrap` для оформления. `json-server` поставлен как `dev-зависимость` — он нужен только при разработке и в собранное приложение не попадает.

Шаблон по умолчанию создаётся с `TypeScript`, поэтому проект переведён на обычный `JavaScript` — язык, на котором велась работа до этого.

Три файла

`package.json` — манифест: имя, скрипты, список зависимостей. `package-lock.json` — точные версии всего дерева зависимостей, чтобы у всех установилось одно и то же. `node_modules` — сами пакеты; папка не коммитится, потому что восстанавливается командой `npm install`. Зависимости делятся на обычные, которые нужны приложению в работе, и `dev-зависимости`, нужные только при разработке.

Этап 2 — слои приложения

Проект разложен по папкам так, как показано в мануале:

- `api` — `instance.js` с экземпляром `axios` и базовым адресом сервера, `events.js` с классом и методами `getAll()` и `createEvent()`, `index.js`, который собирает их вместе;
- `stores` — `index.js` подключает `Pinia` с сохранением состояния, `events.js` описывает состояние со списком мероприятий и действия `loadEvents()` и `createEvent()`;
- `router` — один маршрут, ведущий на страницу мероприятий;
- `layouts` — обёртка `BaseLayout` со `<slot />` для содержимого страницы;
- `views` — страница `EventsPage`;

- **components** — карточка мероприятия **EventCard**.

Компоненты не обращаются к серверу напрямую: страница берёт данные из хранилища, хранилище — из слоя **api**, и только он знает адреса запросов. Поэтому при смене адреса сервера правится один файл.

Зачем разделять слои

Если запросы писать прямо в компонентах, один и тот же адрес расползается по файлам, а при изменении **API** приходится искать все места. Отдельный слой **api** собирает работу с сервером в одном месте, хранилище держит состояние, общее для разных страниц, а компоненты занимаются только отображением.

Этап 3 — возможности Vue

В приложении использованы средства, разобранные на лекции:

- **v-model** — поля формы связаны с данными в обе стороны;
- **v-for** с **:key** — вывод списка карточек;
- **props** — карточка получает название, дату, место и цену извне и ничего не знает о том, откуда они взялись;
- **@submit.prevent** — отправка формы без перезагрузки страницы;
- **mounted** — список загружается при появлении страницы;
- **mapState** и **mapActions** — страница получает список мероприятий и действия прямо из хранилища.

После отправки формы мероприятие уходит на сервер, список перезагружается, а поля очищаются обращением к форме через **\$refs**. Ни в одном месте интерфейс не обновляется вручную.

Реактивность

Данные компонента объявляются в **data**, и Vue запоминает, где они используются в шаблоне, чтобы обновить эти места при изменении значения. Общее состояние вынесено в хранилище, а **mapState** подставляет его в **computed** страницы. Это и есть разница с обычным JavaScript: там пришлось бы вручную находить элементы и переписывать их содержимое при каждом изменении.

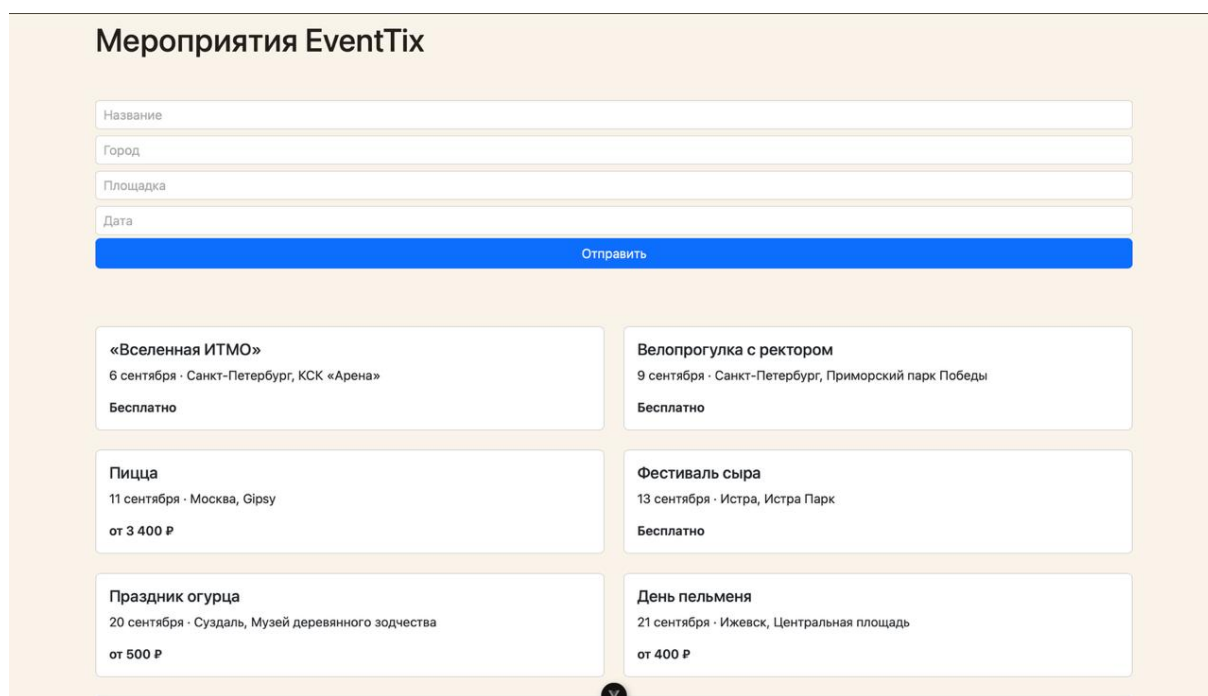


Рис. 1: Приложение в браузере: карточки загружены с сервера, ниже форма добавления.

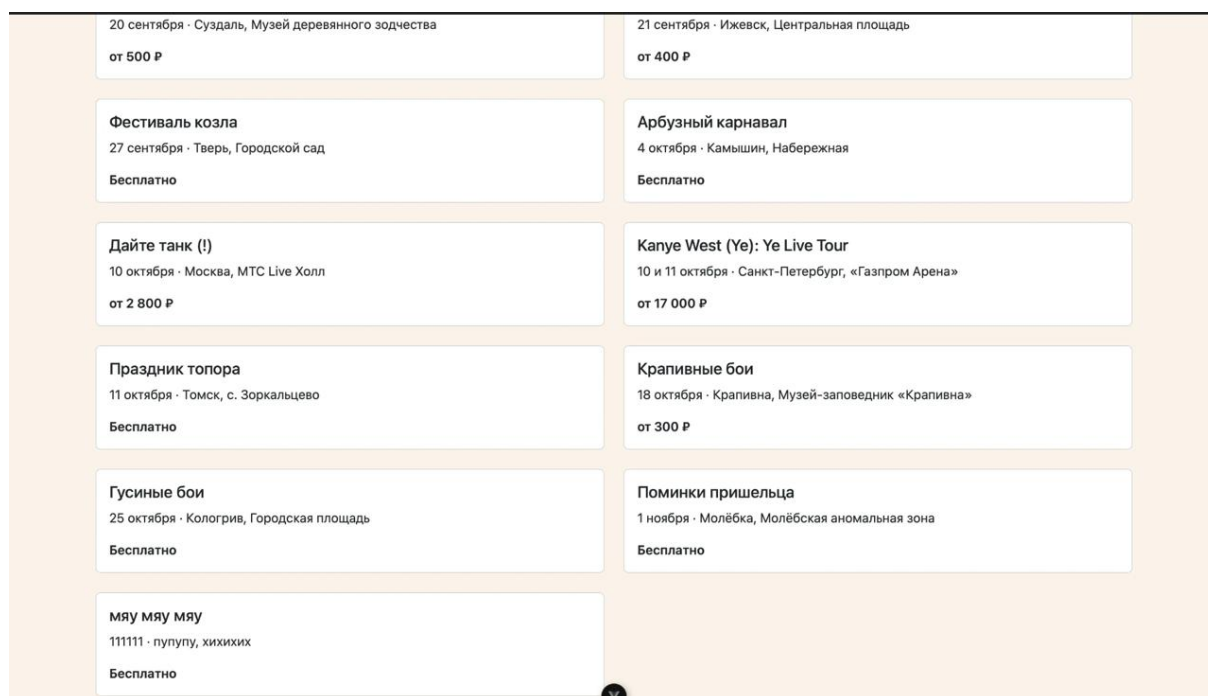


Рис. 2: Мероприятие, добавленное через форму, появилось в конце списка — данные ушли на сервер и вернулись обратно.

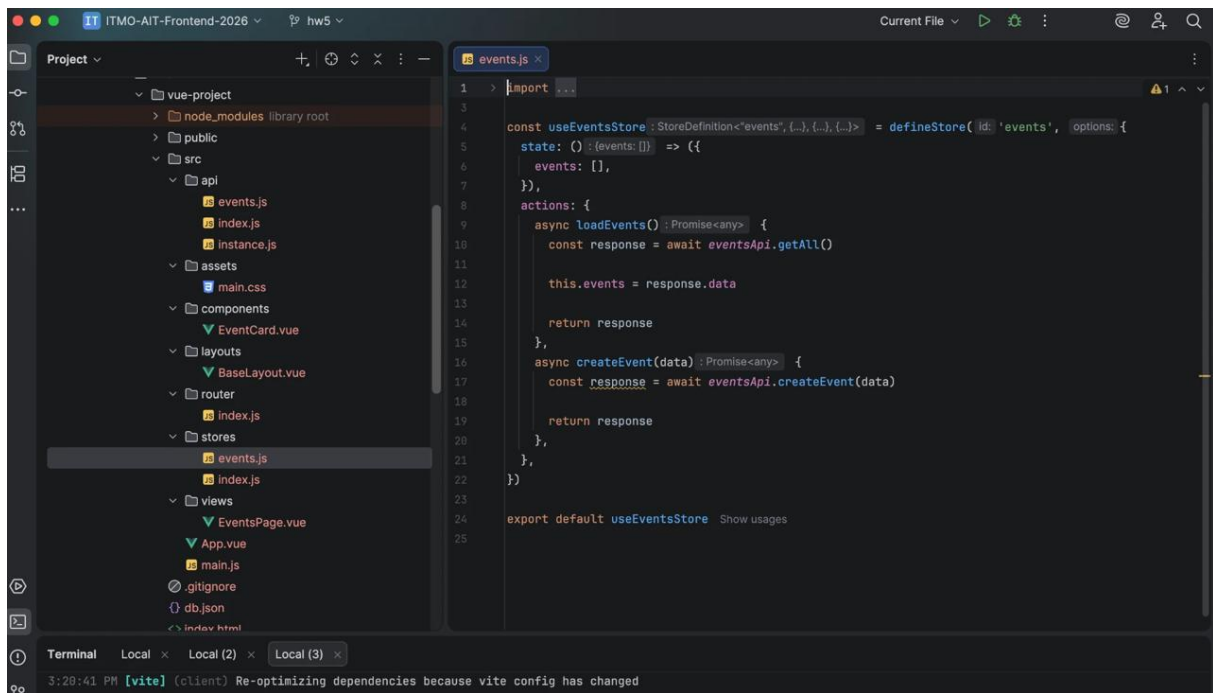


Рис. 3: Структура проекта и код хранилища: состояние со списком мероприятий и действия загрузки и создания.

Вывод

Изучены основные команды `pnpm` и собран проект на Vue со сборщиком Vite, маршрутизацией, хранилищем Pinia и отдельным слоем работы с API. Данные приходят с сервера, а обновление интерфейса выполняет фреймворк.